Software Requirements Specification

for

MedTrack

Prepared by

Group Name: Medheads

Arundas S G	B240147CS	arundas_b240147cs@nitc.ac.in
Haroon Ahmed Sadath K	B240193CS	haroon_b240193cs@nitc.ac.in
Jivitesh Jamuda	B240204CS	jivitesh_b240204cs@nitc.ac.in

Instructor: Dr. A Sudarshan Chakravarthy

Course: CS2011E Database Management Systems

Date: 14-03-2026

Contents

CONTENTS.....

1 INTRODUCTION.....

1.1 Document Purpose.....
1.2 Product Scope.....
1.3 Intended Audience and Document Overview.....
1.4 Definitions, Acronyms and Abbreviations.....
1.5 References and Acknowledgments.....

2 OVERALL DESCRIPTION.....

2.1 Product Overview.....
2.2 Product Functionality.....
2.3 Design and Implementation Constraints.....
2.4 Assumptions and Dependencies.....

3 SPECIFIC REQUIREMENTS.....

3.1 External Interface Requirements.....
3.2 Functional Requirements.....
3.3 Use Case Model.....

4 OTHER NON-FUNCTIONAL REQUIREMENTS.....

4.1 Performance Requirements.....
4.2 Safety and Security Requirements.....
4.3 Software Quality Attributes.....

1 Introduction

1.1 Document Purpose

The purpose of the MedTrack Database Management System is to design and implement an efficient and reliable system for managing medical information within a healthcare organization.

1.2 Product Scope

The scope of the MedTrack Database Management System is to design and develop a relational database that efficiently manages key operations of a healthcare organization. The system will store and organize patient information, doctor details, appointment schedules, prescriptions, and medical records in a centralized database. It will support functions such as appointment booking, maintaining treatment history, and recording prescribed medications. The database will also maintain relationships between different entities such as patients, doctors, and appointments to ensure accurate and consistent data management.

1.3 Intended Audience and Document Overview

The intended audience of the MedTrack Database Management System includes healthcare administrators, doctors, receptionists, and medical staff who are involved in managing patient care and hospital operations. For doctors, the system provides quick access to patient records, medical history, and prescriptions, helping them make informed treatment decisions. Receptionists and administrative staff can efficiently manage appointment scheduling and patient registrations, reducing manual workload and errors. Healthcare administrators benefit from organized and centralized data management, which improves operational efficiency and record-keeping. Overall, the system helps streamline medical workflows, improve accessibility of information, and enhance the quality of healthcare services.

1.4 Definitions, Acronyms and Abbreviations

- **API** - Application Programming Interface
- **CRUD** - Create, Read, Update, and Delete
- **DBMS** - Database Management System
- **EHR** - Electronic Health Record
- **ERD** - Entity-Relationship Diagram
- **HMS** - Hospital Management System
- **SQL** - Structured Query Language
- **SRS** - Software Requirement Specification
- **UI/UX** - User Interface / User Experience

1.5 References and Acknowledgments

- Elmasri, R., & Navathe, S. B. (2015). Fundamentals of Database Systems (7th ed.). Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Retrieved from https://www.postgresql.org/docs/
- Node.js Foundation. (2024). Node.js Documentation. Retrieved from https://nodejs.org/en/docs/
- Vercel. (2024). Next.js Documentation. Retrieved from https://nextjs.org/docs

2 Overall Description

2.1 Product Overview

The **MedTrack Database Management System** is designed to provide an organized and efficient platform for managing medical and administrative data within a healthcare organization. The system focuses on maintaining a centralized database that stores information related to patients, doctors, appointments, prescriptions, and medical records. By structuring this information in a relational database, MedTrack enables quick access, updating, and retrieval of important healthcare data.

The system helps streamline common hospital operations such as patient registration, appointment scheduling, prescription management, and maintaining medical histories. It establishes relationships between different entities like patients, doctors, and appointments to ensure accurate and consistent data management. MedTrack also aims to reduce data redundancy, improve record accuracy, and support better coordination between medical staff and administrative personnel.

2.2 Product Functionality

1. Patient Management

This module manages all information related to patients.

- Register new patients
- Store personal details (name, age, gender, address, phone)
- Maintain medical history
- Record allergies and chronic conditions
- Track previous visits and treatments

2. Doctor Management

This functionality manages doctor-related information.

- Store doctor details
- Record specialization (cardiology, neurology, etc.)
- Assign doctors to departments
- Maintain doctor schedules and availability

3. Appointment Scheduling

This module handles booking and managing appointments.

- Book appointments with doctors
- View doctor availability
- Cancel or reschedule appointments
- Track appointment status (scheduled, completed, cancelled)

4. Prescription Management

This module records medicines prescribed during consultations.

- Create digital prescriptions
- Store medicine name, dosage, and duration
- Record doctor notes and instructions
- Maintain prescription history

5. Electronic Medical Records (EMR)

This module stores a patient's medical history and treatment details.

- Diagnosis records
- Treatment plans
- Previous medical conditions
- Doctor observations
- Follow-up recommendations

6. Laboratory Test Management

This functionality manages lab tests and reports.

- Order lab tests during consultation
- Store test results
- Attach digital reports
- Track test history

7. Pharmacy / Medicine Inventory

This module manages medicines available in the pharmacy.

- Store medicine details
- Track stock quantity
- Update medicine inventory
- Generate low-stock alerts

8. Billing and Payment Management

This functionality handles financial transactions.

- Generate patient bills
- Record consultation fees
- Add charges for lab tests and medicines
- Track payment status

9. Staff Management

This module manages non-doctor staff in the hospital.

- Store staff details
- Assign roles (nurse, receptionist, admin)
- Track staff responsibilities

10. Notification and Reminder System

This module sends alerts and reminders.

- Appointment reminders
- Follow-up reminders
- Medicine reminders

11. Reporting and Analytics

This module provides statistical insights.

- Daily patient count
- Doctor workload reports
- Revenue reports
- Most prescribed medicines

2.3 Design and Implementation Constraints

- The system ought to function on standard PCs or servers that the college uses. It must function properly on standard hardware without requiring really powerful computers.
- The system will employ PostgreSQL for data storage, Node.js for server management (back-end), and Next for front-end website development. For this project, these technologies have been selected.
- Git will be used by the system to coordinate teamwork and track code changes.
- To develop the system, we shall employ the COMET approach.
- To produce visual diagrams that illustrate how various system components function and communicate with one another, we will utilize UML.
- The Internet connection is a constraint for the application. Since the application fetches data from the database over the Internet, there must be an Internet connection for the application to function.

2.4 Assumptions and Dependencies

- Stable Internet Connection – The users are expected to have a reliable internet connection for seamless access and updates.
- Browser Compatibility – The application will work on modern web browsers but may not support outdated versions.
- Third-Party Services/Dependencies – The project depends on external authentication providers and cloud services, assuming they remain available and functional.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

There are five different types of interfaces for the MedTrack system based on user roles:

Doctor Interface:

- **Login**: Doctors log in using their Doctor ID and Password.
- **Navigation**: A dashboard displays menu options like Patient Records, Daily Schedule, and Prescriptions. Navigation is done through clickable buttons.
- **Interaction Style**: Doctors interact through medical forms, dropdowns for diagnoses, and text fields for treatment notes.
- **User Input**: Doctors input diagnosis details, order lab tests, and generate prescriptions.

Patient Interface:

- **Login**: Patients log in using their Patient ID and Password.
- **Navigation**: A dashboard displays options like Book Appointment, My Medical History, and Prescriptions.
- **Interaction Style**: Patients interact using calendar widgets to pick appointment dates, dropdowns to select departments, and clickable tables to view lab reports.
- **User Input**: Patients input personal contact details and appointment preferences.

Administrator Interface:

- **Login**: Admins log in using their Admin ID and Password.
- **Navigation**: A dashboard displays options like Manage Staff, Department Records, and System Analytics.
- **Interaction Style**: Admins interact with data grids, search fields, and management modules.
- **User Input**: Admins input staff details for registration, system configuration data, and permission updates.

Receptionist Interface:

- **Login**: Receptionists log in using their Receptionist ID and Password.
- **Navigation**: A dashboard displays options like Patient Registration, Doctor Availability, and Appointment Scheduling.
- **Interaction Style**: Interaction happens through scheduling calendars, search bars to find patients, and fast-entry forms.
- **User Input**: Receptionists input walk-in patient details and manual appointment modifications.

Pharmacist Interface:

- **Login**: Pharmacists log in using their Pharmacist ID and Password.
- **Navigation**: A dashboard displays options like Pending Prescriptions, Medicine Inventory, and Dispense Logs.
- **Interaction Style**: Interaction through search bars (by Patient ID or Prescription ID) and inventory tables.
- **User Input**: Pharmacists input inventory updates and mark prescriptions as dispensed.

Lab Technician Interface

- **Login**: Lab Technicians log in using LabTech ID and Password.
- **Navigation**: Dashboard shows options like Pending Lab Tests, Upload Reports, Test History.
- **Interaction Style**: File upload forms, report entry forms, and patient search.
- **User Input**: Upload lab test results (PDF/image), enter test values, and update test status.

3.1.2 Hardware Interfaces

- **Supported Devices**: The system will be accessible through web browsers, making it compatible with desktops, laptops, tablets, and mobile devices at the hospital.
- **Input Devices**: Users will interact with the system using keyboards, mice, or touchscreens.
- **Servers**: The backend will run on a cloud or local hospital server, processing requests and handling database operations.
- **Network Interfaces**: The system will communicate securely over the internet or internal hospital intranet via HTTPS.

3.1.3 Software Interfaces

- The primary software component is the web interface where all users interact.
- The web interface communicates with the backend server, which processes queries using the relational database.
- The frontend and backend exchange data securely via HTTPS requests.

3.2 Functional Requirements

- **F1**: The system shall allow users to log in using their respective Role IDs and Passwords.
- **F2**: The system shall differentiate users based on their login credentials and enforce role-based access control.
- **F3**: The system shall allow Receptionists and Patients to register new patient profiles.
- **F4**: The system shall allow Patients to book, reschedule, or cancel appointments with available Doctors.
- **F5**: The system shall allow Doctors to view their daily appointment schedule.
- **F6**: The system shall allow Doctors to access patient medical history and previous lab reports.
- **F7**: The system shall allow Doctors to create and save medical prescriptions to a patient's record.
- **F8**: The system shall allow Pharmacists to view active prescriptions issued by Doctors.
- **F9**: The system shall allow Pharmacists to update medicine inventory after dispensing.
- **F10**: The system shall allow Administrators to add, update, or remove staff and doctor profiles.
- **F11**: The system shall allow Receptionists to view doctor availability in real-time.
- **F12**: The system shall send appointment reminders to Patients.

3.3 Use Case Model

3.3.1 Use Case #1: Book Appointment (U1)

- **Purpose**: Allow patients or receptionists to book an appointment with a doctor.
- **Requirements Traceability**: Patients should be able to view doctor availability and reserve a time slot.
- **Priority**: High

Preconditions:

- The Patient or Receptionist must be logged in.
- The Doctor must have available time slots in the schedule.

Postconditions:

- The appointment is saved in the database.
- The doctor's availability is updated.
- Actors: Patient / Receptionist (Primary)

Basic Flow:

- User navigates to the Appointment Booking page.
- User selects the desired Department and Doctor.
- Application displays available dates and times.
- User selects a time slot and confirms.
- Application saves the appointment and displays a confirmation message.

Exceptions:

If another user books the exact time slot simultaneously, the application prompts the user to select a new time.

3.3.2 Use Case #2: Create Prescription (U2)

- **Purpose**: Allow doctors to prescribe medicine and record treatment notes.
- **Requirements Traceability**: Doctors should be able to assign medicines and dosages to a specific patient appointment.
- **Priority**: High

Preconditions:

- Doctor must be logged in.
- An appointment record must exist for the patient.
- Postconditions:
- The prescription is stored in the database.
- The Pharmacist and Patient can now view the prescription.
- Actors: Doctor (Primary)

Basic Flow:

- Doctor opens the Patient's current appointment record.
- Doctor enters the diagnosis and treatment notes.
- Doctor searches for medicines and adds dosages to the prescription.
- Doctor clicks submit.
- Application saves the data and displays a success message.

Exceptions:

If a selected medicine is completely out of stock in the hospital pharmacy, the application may display a warning flag.

3.3.3 Use Case #3: Dispense Medicine (U3)

- **Purpose**: Allow pharmacists to fulfill a doctor's prescription and update inventory.
- **Requirements Traceability**: Pharmacists must see valid prescriptions and deduct given stock from the database.
- **Priority**: High

Preconditions:

- Pharmacist must be logged in.
- A valid, unfulfilled prescription must exist in the system.

Postconditions:

- Prescription is marked as "Dispensed".
- Medicine inventory counts are reduced.
- Actors: Pharmacist (Primary)

Basic Flow:

- Pharmacist searches for the Patient ID or Prescription ID.
- Application displays the prescribed medicines.
- Pharmacist physically gathers the medicines and clicks "Mark as Dispensed".
- Application deducts the quantities from the inventory database.

Exceptions:

If the required medicine quantity exceeds the current stock, the system alerts the Pharmacist to order more or partially fulfill the prescription.

3.3.4 Use Case #4: Upload Lab Test Results (U4)

- **Purpose**: Allow lab technicians to record and upload laboratory test results for patients so that doctors and patients can access them through the system.
- **Requirements Traceability**: Lab technicians must be able to view pending test requests and update them with test values and reports.
- **Priority**: High

Preconditions

- The Lab Technician must be logged into the system.
- A doctor must have previously ordered a lab test for the patient.
- The patient and test record must exist in the database.

Postconditions

- The lab test results are stored in the database.
- The test status is updated to Completed.
- The doctor and patient can view the test results through their interfaces.
- Actors: Lab Technician

Basic Flow

- Lab Technician logs into the system.
- Technician navigates to the Pending Lab Tests section.
- The system displays a list of pending test requests.
- Technician selects a patient test record.
- Technician enters the test values and observations.
- Technician uploads the laboratory report (PDF/Image).
- Technician submits the result.
- The system saves the data and updates the test status to Completed.

Exceptions

- If the uploaded file format is unsupported, the system prompts the technician to upload a valid format.
- If the technician tries to submit incomplete test data, the system displays an error and requests the missing information.

4 Other Non-functional Requirements

4.1 Performance Requirements

Performance should not be as issue as:

- The system is designed to handle lightweight queries, ensuring quick data retrieval and minimal server load.
- UI transitions and screen changes will be smooth, requiring minimal computation.
- Server updates should complete within a few seconds, provided a stable internet connection is maintained.

4.2 Safety and Security Requirements

- All user data must be securely stored and encrypted to prevent unauthorised access.
- The system must enforce authentication using JWT for secure access.
- Communication between the frontend and backend should be encrypted using HTTPS.
- Role-Based Access Control (RBAC) will be implemented to restrict user permissions.

4.3 Software Quality Attributes

4.3.1 Performance Requirements

- The system should handle concurrent users efficiently, ensuring smooth performance.
- Queries to the database should be optimised to minimise latency.
- The user interface should be responsive and provide a seamless user experience.

4.3.2 Maintainability and Scalability

- The codebase should follow reusable design principles to simplify maintenance.
- Proper documentation and comments should be provided to facilitate future maintenance and enhancements.
- A user manual will be provided to guide users on system functionalities and usage.